Chapter 18

EXTENSIONS

The procedures of GWB form a logically complete whole in that they allow solids to be created, processed, and stored in disk files. Nevertheless, to make GWB also practically interesting, enhancements as for functionality and efficiency would be needed. This chapter outlines these extensions to the basic GWB.

18.1 EXTENSION OF GEOMETRIC COVERAGE

The geometric coverage of the basic modeler constructed in the second part of this book is not sufficient for tasks where exact modeling of curved surfaces is needed. Hence the extension of the modeler to handle other than merely planar surfaces would be valuable.

Unfortunately, it is difficult to build a solid modeler capable of processing a wide variety of surfaces, and we do not attempt to discuss in detail the construction of a full-scale modeler with curved surfaces. Instead, we describe a much simpler extension that enhances the functionality of the basic modeler while requiring only a modest effort.

Recall that the basic modeler is capable of creating solid objects as parameterized primitives such as blocks, cylinders, and tori. In a sense, the information that a certain object was originally created in this fashion is lost, however: the data structures used have no means of representing that the faces approximating the cylinder surface in Figure 18.1 logically belong together.

Due to this lack of information, the best we can do to render a cylinder is represented in (a), (b), or (c) in the figure. In order to produce images such as (d), (e), the least we need is the capability of telling whether an edge separates two faces arising from distinct surfaces or not.

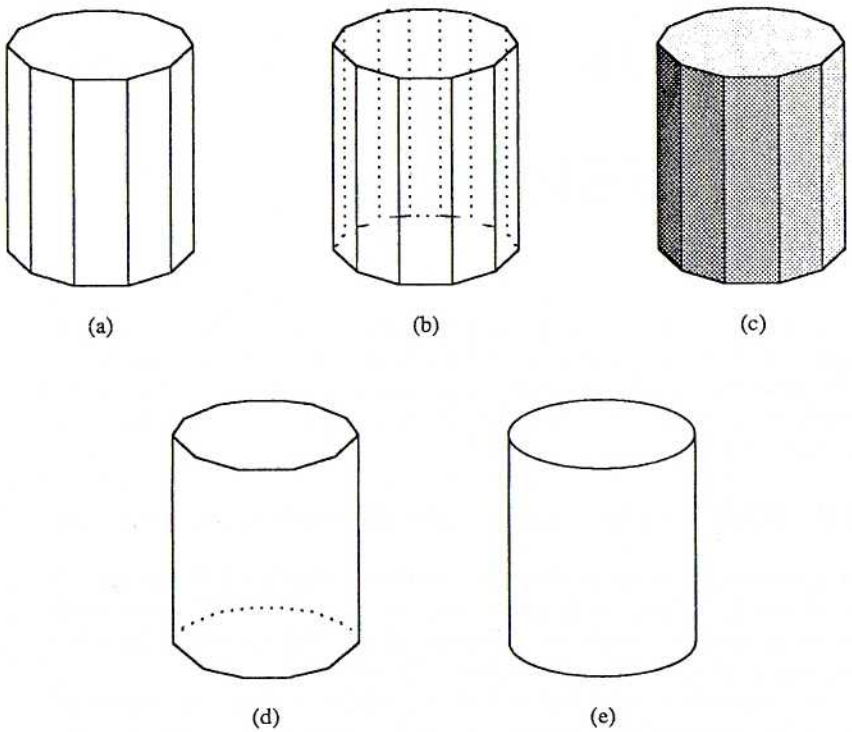

Figure 18.1 Graphical renderings of a cylinder.

The loss of information causes problems in functionality besides just bad pictures. As we have no concept for distinguishing the set of edges forming an approximation of the intersection curve between the top plane and the cylinder surface, we cannot easily specify operations we might want to perform on this curve as a whole—say, to replace it with a toroidal blending surface.

The solution to these problems is to expand the data structure used in the basic modeler by including concepts for representing the surface information, and by modifying all necessary algorithms to deal properly with this data.

18.1.1 Representation of Surfaces

The class of surfaces we are potentially interested in includes all types of surfaces ordinarily found in CSG schemes—that is, natural quadrics and tori. We also want to design the extension so that parametric surfaces can be accommodated, if desired.

One of the leading ideas of boundary representations is the separation of topological and geometrical information. Following this approach, we choose to represent surfaces in "infinite" geometry. For instance, a cylinder surface is represented as an infinite object; the part(s) of it really belonging to object(s) are "chalked out" by edges of the basic modeler. Hence, every surface will be represented by a tuple of parameters that describe it in some convenient orientation, and a transformation matrix that maps it into its true position.

For basic half-spaces, the following parameters could be used:

Plane: The four parameters a, b, c, and d of the plane equation

$$[a\ b\ c\ d] \cdot [x\ y\ z\ w]^T = 0$$

need to be stored. Planes are exceptional in that they do not need an explicit transformation matrix; instead, the parameters can be transformed as if they formed the homogeneous coordinates of a point.

Cylinder: Cylinders are represented as if their axis were the z-axis, and their bottom face on the xy-plane. Parameters needed are the radius r and the height h of the cylinder.

Cone: Cones are represented similarly as cylinders.

Sphere: Spheres are represented as if they were centered at origin. Radius r is the only parameter needed.

Torus: Tori are represented as if they were centered at the origin and their axis was along the z-axis. Parameters needed are the smaller and larger radii r_s, r_l.

Program 18.1 outlines the necessary changes in the boundary data structure. The "times-used" counter is necessary for proper memory management.

18.1.2 Generation of Surface Information

All procedures of the basic modeler that create faces must be modified so as to deal properly with the new surface information. These include primitives, sweeps, transformations, set operations, splitting, and the undo mechanism.

Most of these modifications are straightforward. Primitives must be extended so as to create surface nodes as required, and assign surface pointers of faces properly. Transformation routines must also transform surfaces. Set operations and splitting must copy the surface information whenever they divide a face into two faces.

Proper changes of the undoing procedures are slightly more tricky. First, undoable procedures for the creation and the deletion of surfaces must be written. For instance, all cylinders would be generated by means of a procedure

```
Cylinder *newcylinder(r, h)
float    r, h;
```

which must be undoable. Hence, the procedure would store the arguments of the corresponding deletion procedure

```
void     delcylinder(c)
Cylinder *c;
```

and vice versa.

Second, the assignment of surface information to polyhedral faces must be made undoable. That is, surface information should be assigned by means of a procedure

```
void     *setsurf(f, s)
Face     *f;
Surface  *s;
```

that must record the previous value of the surface hook into the undo log. The procedure should be written so as to maintain the *times_used* counter;


```
typedef struct
{
        short       surf_type;        /* == PLANE */
        short       times_used;
        real        a, b, c, d;
} Plane;

typedef struct
{
        short       surf_type;        /* == CYLINDER */
        short       times_used;
        real        cy_transf[4][4];
        real        cy_rad;
        real        cy_h;
} Cylinder;

...

typedef union
{
        Plane       p;
        Cylinder    cy;
        Cone        co;
        Sphere      sph;
        Torus       t;
} Surf;

struct face
{
        Id          faceno;           /* face identifier */
        ...
        Surf        *fsurf;           /* surface information */
};
```

Program 18.1 Data structures for surface representation.

if the counter of a surface drops to zero, the procedure can delete the surface node by means of the appropriate deletion procedure.

Finally, those Euler operators that delete faces (KEF, KFMRH, KVFS) must be modified so as to clear the surface information of the face by means of `setsurf`. In addition, the procedure `applyop` of Program 16.8 must be extended so as to handle the basic operations related to surfaces as well.

After these changes, undoing and all related procedures (such as copying and inversion) will deal properly with surface information.

Extended Sweeps

Sweeps are the procedures that require the most changes, because we need to mark the faces created by linear sweeping of a sequence of edges approximating a circular arc as belonging to a cylinder surface. But this cannot be done without extending the basic data structure, as it has no concept for circular arcs.

This information can be added in strict analogy to the case of surface information. Hence, each edge will have a pointer to a curve node that includes the sufficient information. A design for straight lines and circular arcs is outlined in Program 18.2. Observe that straight lines do not need explicit information, as their endpoints are available in the boundary data structure through half-edges. As arcs are plane curves, they are represented with the natural parameters and a pointer to the plane they lie on.

With this additional information, the linear sweeping algorithm can examine the curve pointers of edges of the face swept, and tag the faces as planes or cylinders. Similarly, the rotational sweeping algorithm can tag faces generated from straight profile lines as cylinders or cones, and faces generated from arcs spheres or tori.

In order to preserve undoing, curve information should be manipulated by means of procedures similar to the procedures dealing with surface information.

18.1.3 Functionality with Surface Information

The addition of surface information into the data structure readily gives the information needed for better image generation. For instance, shading according to the Phong or Gouraud models requires that surface normals are calculated at vertices of a mesh of polygons. Now this information can be formed.

Besides graphics, the ability of grouping related entities of the boundary data structure together has many other uses. The blending of two surfaces was already mentioned in the above. Another wide group of facilities is


```
typedef struct
{
        short       curve_type;       /* == LINE */
        short       times_used;
} Line;

typedef struct
{
        short       curve_type;       /* == ARC */
        short       times_used;
        real        arc_rad;          /* radius */
        real        arc_cx, arc_cy;   /* center */
        real        arc_phi1;         /* start angle */
        real        arc_phi2;         /* end angle */
        plane       *arc_plane;       /* plane of the arc */
} Arc;

...

typedef union
{
        line        l;
        arc         a;
} Curve;

struct edge
{
        HalfEdge    *he1;
        HalfEdge    *he2;
        Curve       *ecurv;           /* curve information */
        ...
};
```

Program 18.2 Data structures for curve representation.

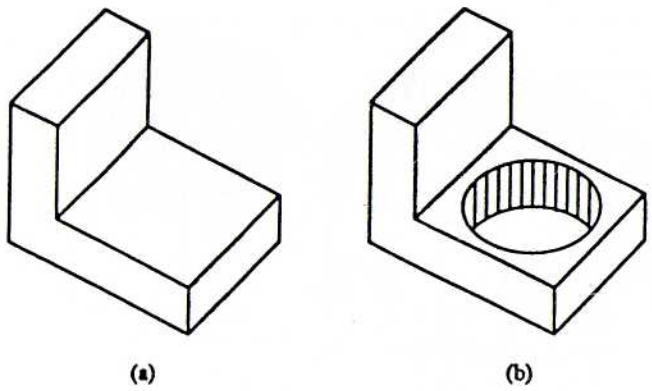

Figure 18.2 L-brick with a cylindrical hole.

opened by the concept of *feature extraction* which is made possible by the new information. In general, a "feature" is an interrelated group of geometric entities that has significance for some particular problem—say, the set of edges separating two surfaces for the blending problem.

Feature recognition plays a central role in advanced applications of solid modeling. Consider, for instance, the familiar object depicted in Figure 18.2. A program performing planning of machining operations needs the capability of recognizing that a certain set of faces, edges, and vertices forms a "cylindrical hole" in order to decide what kinds of machining operations will modify solid (a) to solid (b).

Model-based *image recognition* gives another case for feature extraction. In this application, features found from a binary image must be related to features extracted from a geometric model, for instance to determine which object is seen and to calculate its orientation.

18.2 EFFICIENT GEOMETRIC SEARCH

Solid modeling algorithms are computationally intensive. One of the major challenges of the designer of a solid modeler is to include appropriate mechanisms for speeding up the solid modeling algorithm to an acceptable level.

The general theoretical study of efficient geometric algorithms is the area of *computational geometry*. For good introductions to computational geometry, see [48,110,111,67]. Nevertheless, workers in computational geometry usually pursue good worst-case asymptotic efficiency by means of advanced data structures and algorithms, while a designer of a solid modeler has much less freedom in the selection of data structures, and must also be concerned with the "practical" expected efficiency of his/her algorithms. Therefore, results of computational geometry are not always applicable to him/her.

This section describes some of the alternative approaches available to the designer of a solid modeler who is concerned on the speed of his/her modeler. For reasons of clarity, we shall consider the prime example of Boolean set operations on solids.

As typical for algorithms of solid modeling, a major factor in the computational cost of set operations is caused by the search of potentially intersecting geometric objects. For instance, the algorithm of Chapter 15 works by comparing all m edges of one solid against the n faces of the other solid. Hence, a total of mn edge-to-face comparisons is involved. While the average cost of per comparison can be made smaller by calculating the enclosing boxes of the two solids and all faces, the mn term will still dominate the asymptotic cost of a set operation.

From the point of view of asymptotic optimality, there is not very much that can be done about this: two polyhedra of sizes m and n can have as many as $O(mn)$ pairs of intersecting faces. In practice, however, cases that really involve a quadratic number of intersections are rare. Much more often the "size" of the overlap is a linear function of n and m rather than quadratic. Observe that two *convex* polyhedra of sizes n have no more than $O(n)$ intersections.

Another typical feature of "practical" cases is that the effect of a set operation is *localized* in the three-dimensional space E^3. That is, set operations add or remove material in some fairly restricted part of the space, and leave must of the geometric data untouched. This situation is typical for the interactive use of set operations for design.

A "practical" set operations algorithm should be able to take advantage of these kinds of locality. If the search for intersections can be localized to the area where they are likely to occur, the set operations algorithm should for all practical purposes be capable of working in linear expected time as a function of the size of the inputs.

18.2.1 Geometric Queries and Indices

The basic computational step of the set operations algorithm (*"given a query edge (ray), find all faces (boxes) that intersect it"*) typifies the more general problem of providing *access by location* in geometric algorithms. Other examples are listed below:

1. *Box Query*: Given a box, report all geometric entities that intersect it.

2. *Range Query*: Report all geometric entities that are within some distance r from a query location x, y, z.

3. *Nearest Neighbors Query*: Report n nearest geometric entities from a query location x, y, z.

We call these and identical tasks *geometric queries*. A *geometric index* is defined as a data structure that enhances the efficiency of geometric queries.

Ordinary access methods (B-trees, hashing, etc.) developed for data base management systems are not applicable for geometric tasks, because coordinate values can seldom be utilized as exact keys in access by location. Processing of things with geometric extent is also problematic.

18.2.2 Hierarchic Indices

Hierarchic object descriptions belong to the standard techniques of computer graphics for representing complex objects [29]. Indeed, when objects are assembled from transformed instances of other objects as outlined in Section 2.2.2, a hierarchy naturally arises. Octrees of Section 4.2.1 and CSG trees of Section 5.2 are other examples of hierarchic object descriptions we have encountered so far.

The tree structure of a hierarchic object description suggests some obvious mechanisms for speeding up geometric queries. We might include an enclosing box to each node of the tree, i.e., indicate the corners of the smallest rectangular block that completely contains the object represented by the subtree rooted at that node. With this information, the *pruning* of complete subtrees becomes possible. For instance, when processing a ray query, we can test for the intersection of the ray and the enclosing box before looking into a subtree. If the ray does not hit the enclosing box, we can ignore that subtree immediately; see Figure 18.3. Needless to say, this mechanism is ubiquitous in ray casting. It can also be extended to other algorithms, in particular boundary evaluation [119].

Hierarchic indices are most natural when the object description itself is naturally a hierarchy as in the examples listed above. In this case little additional data structures are needed for representing the hierarchic access structure. Nevertheless, hierarchic indices can be applied also when the object description is not hierarchic itself.

In fact, the half-edge data structure described in Chapter 10 includes an elementary hierarchic geometric index. According to Program 10.1, nodes *Solid* and *Face* of the data structure have a "hook" for associating

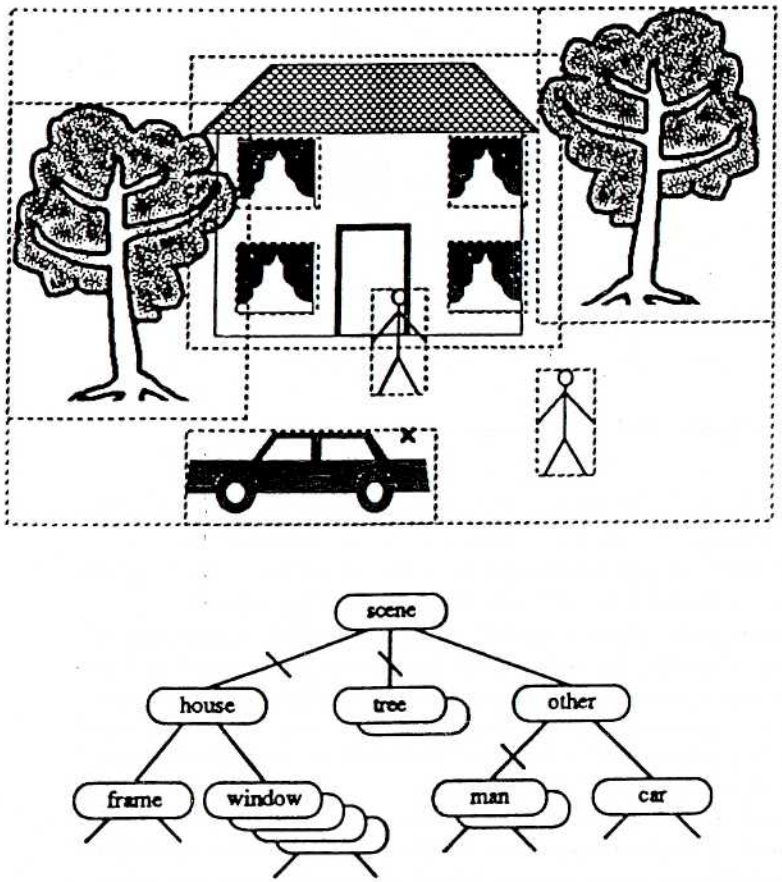

Figure 18.3 Pruning a hierarchic object description.

an enclosing box with the node. Even this extremely simple mechanism has a profound effect to the speed of algorithms that do lots of geometric search, such as the set operations algorithm.

Recall that the geometric search of the set operations algorithm takes the form of a face-face comparison. Clearly, the information of enclosing boxes can be used to speed up the search of potentially intersecting face pairs. As we have a two-level hierarchy, we can first compare a face against the enclosing box of the other solid; face boxes need to be compared only if this initial test indicates a potential intersection. Usually most potential intersections can be rejected by box intersection tests.

BUILD-2 [18] goes further in this respect, and allows for hierarchies of unlimited depth. In this case, "bubble neighborhoods," i.e., spherical enclosures represented in terms of a center point and a radius, are used instead of rectangular boxes. A special node of the data structure, the "face party," is used to represent the hierarchic index. The parties form a tree rooted at the root node of the data structure having faces at the leaves. Usually faces of a single party have some kind of a logical connection, e.g., they come from the same primitive.

18.2.3 Grid Indices

When the object description naturally takes the form of a hierarchy, hierarchic indices are very attractive. However, if this it not the case, there are some drawbacks. In particular, when used for boundary models, the speed gains may deteriorate when the object is modified by algorithms such as set operations and splitting. If the face hierarchy has no correspondence with the spatial distribution of faces, box tests bring little help. Moreover, the rearrangement of the hierarchy so as to make it more useful is itself a complicated geometric problem that definitely requires efficient geometric access.

Grid indices form another widely used family of geometric indices that avoids many of the problems of hierarchic indices. They are based on subdividing the space of interest into a set of nonoverlapping *cells*. Each cell is represented by one or several *data pages* that contain (references to) all geometric entities (points, faces) that intersect the cell. A directory mechanism provides for fast access to cells by location.

Regular Grid Indices

Regular grids are the simplest example of grid indices. In this approach, the space of interest is subdivided into a regular grid of rectangles or boxes

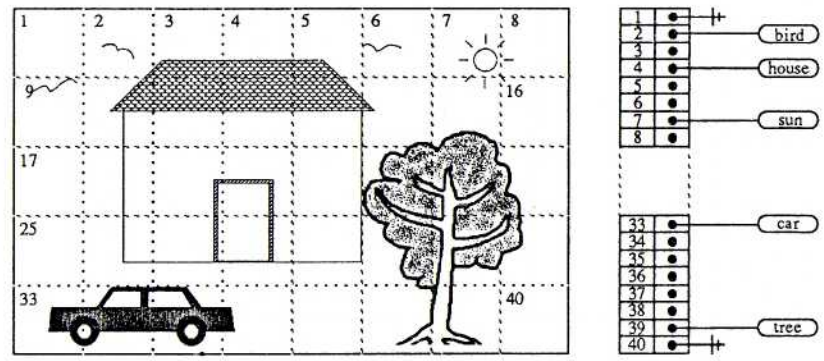

Figure 18.4 Regular grid index.

that can be thought of as an two- or three-dimensional array of cells. Hence geometric access to cells amounts to simple index calculation.

As all grid indices, regular grids can be used with all kinds of geometric information. For instance, TIPS [90] uses a regular grid as an access mechanism to the half-spaces of a model. A boundary modeler can use a grid index to faces by storing a reference to a face into each cell intersected by the face or its enclosing box.

Extendible Cell Indices

A drawback of regular grids is that they do not cope very well if the spatial density of the data is highly variable. In the extreme case, most cells would be empty, while the few nonempty cells would need many overflow pages to store all objects that intersect them. Hence, the overall performance deteriorates to that of the exhaustive search.

One way to overcome these difficulties is to let the size of individual cells vary according to the spatial density of data. This approach leads us to an *extendible cell* structure.

The EXCELL Method Let us discuss a particular extendible cell mechanism, the EXCELL method [116], in more depth. An EXCELL structure consists of two parts: a *data part* and a *directory*. The cells of the data part are formed by recursively halving the space of interest until the data corresponding with that part of the space fits into a data page. The directory is an array that provides efficient access by location to data pages.

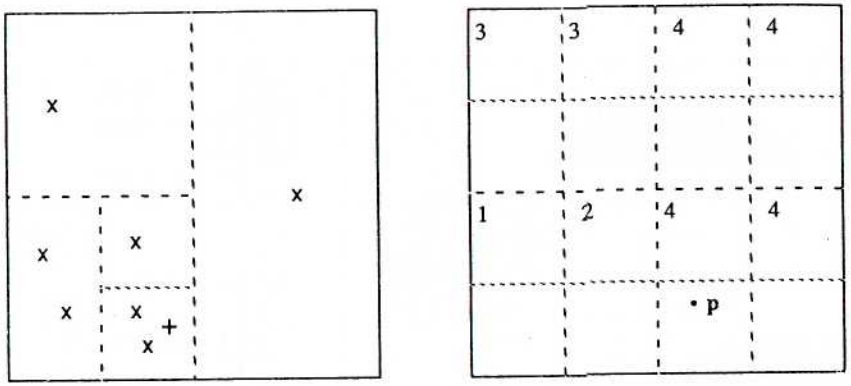

Figure 18.5 EXCELL concepts.

Figure 18.5 clarifies these concepts. In the figure, the two-dimensional space of interest is divided into four cells (rectangles), none of which contains more than three (= the data page capacity b) points. The directory is an array of elements each corresponding to a rectangle of minimal size; each element contains a pointer to the corresponding data page. For instance, the cell containing the query point p is found by calculating the directory array index from the coordinates of p, and then fetching the data page pointed at by that array element. The insertion of a new point (+) would require halving of the cell because of overflow; as this cell was of minimal size, the directory elements must be halved also, hence doubling the size of the directory array.

When applied to structured geometric objects (lines, rectangles), EXCELL cells contain pointers to the entities (potentially) intersecting it. Arbitrary objects can be stored by using pointers to enclosing boxes. In these cases, the halving process is controlled by the maximal number of pointers fitting in a data page.

The simple versions of EXCELL described above will fall apart if more than the maximum cell capacity of b objects exactly meet each other. This will happen, for instance, if more than b enclosing boxes intersect each other—an event which is particularly likely if a three-dimensional generalization of the index is used. To overcome this problem, a two-level hierarchy mechanism and overflow capability should be included to the basic index. Figure 18.6 depicts an example of a hierarchic EXCELL structure.

A geometric index should be an easily accessible tool that can be exploited readily in algorithms that need to perform geometric search. In the

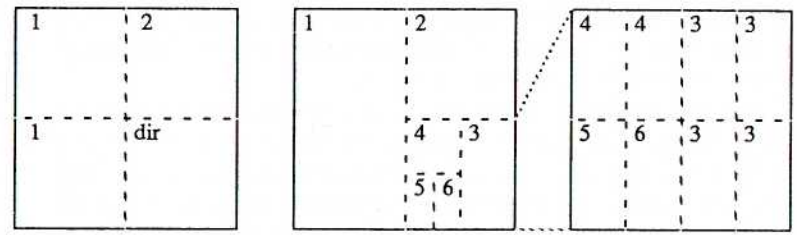

Figure 18.6 Hierarchic EXCELL.

case of EXCELL, this is possible by selecting a suitable interface between the algorithm and EXCELL.

For maximum flexibility, the index should be capable of storing both points (for storing references to objects having location, but no geometric extent) and boxes (for objects that have geometric extent). The functionality of such a geometric index is represented in the Table 18.1. In particular, the "query object" *cursor* of this design can be a point, a ray, or a box, and the queries supported can include all the types listed above.

Set Operations With EXCELL Let us outline how EXCELL could be used to enhance the set operations algorithm of Chapter 15. The obvious way to do this is to replace the exhaustive search for intersections with the edge query supported by EXCELL.

Program 18.3 outlines in C-like pseudocode the implementation of the geometric search for intersections with EXCELL. The code fragment creates an index for one of the argument solids of the set operation, and uses it to find potentially intersecting face pairs. The algorithm assumes that box nodes stored in EXCELL are similar to the box nodes of the half-edge data structure.

If a solid appears as an argument of a sequence of set operations, an index for it may be generated several times. A natural enhancement of the algorithm of Program 18.3 avoids this by constructing the index just once, and by updating it as the final phase of each set operation.

The algorithm of Program 18.4 outlines the index updating operation. The update rules are analogous to the assembly rules for producing the result of the set operation from the boundary classification; for instance, in the case of set union, faces of *ainb* are removed from the geometric index, and faces of *bouta* are inserted to it. Faces split during the set

`index *createindex()`

Create geometric index, return a pointer to it.

`insertindex(root, obj, type)`

Insert object *obj* of type *type* into the index.

`removeindex(root, obj)`

Remove object *obj* from the index.

`deleteindex(root)`

Delete geometric index.

`openquery(root, cursor, type)`

Open geometric query of type *type* with query object *cursor*.

`objtype *nextquery()`

Return the next resulting object of an open query.

Table 18.1 EXCELL index functions.


```
root = createindex();
fa = sa->sfaces;
while(fa)
{
        if(fa intersects sb->sbox)
                insertindex(root, fa->fbox, BOX);
        fa = fa->nextf;
}
eb = sb->sedges;
while(eb)
{
        if(eb intersects sa->sbox)
        {
                openquery(root, e, EDGE_QUERY)
                while((b = nextquery()) != NIL)
                {
                        process(face of b, eb);
                }
        }
}
```

Program 18.3 Implementation of the search in set operations.

```
switch(op)
{
        case UNION:
                for(each face f of ainb)
                        removeindex(root, f->fbox);
                for(each face f of bouta)
                        insertindex(root, f->fbox, BOX);
                for(each face f in aoutb adjacent
                    to the intersection polygons)
                {
                        removeindex(root, f->fbox);
                        recalculate f->fbox;
                        insertindex(root, f->fbox, BOX);
                }
        case INTER:
                ...
        case MINUS:
                ...
}
```

Program 18.4 Index updating in set operations.

operation (which are available by scanning the intersection polygons) must be handled separately. As their geometric extent has changed, a new box should be calculated. In fact, some of them may have no box at all in the index because of the removal of faces in *ainb*.

Experimental results [79] indicate that the enhanced set operations work in approximately linear time in the size of their inputs. Moreover, by virtue of the updating operation, the time needed for repeated set operations on a large model appears to depend only on the size of the "smaller" arguments and the size of the geometric overlap.

In contrast to many theoretical results of computational geometry, calling for very difficult data structures and algorithms, EXCELL is relatively simple to implement. Yet it seems to provide qualitatively the kind of performance enhancement we were originally interested in.

Quite obviously, EXCELL can be used for many other tasks besides the set operations. Ray casting is the most obvious example. In fact, EXCELL can determine the first intersected face (or any other object) along the ray in *constant* expected time. Hence with EXCELL, the computation needed the create a ray-casted image of a scene does not (asymptotically) depend on its complexity but only on the desired resolution.

BIBLIOGRAPHIC NOTES

The representation and the processing of general curved surfaces is beyond the scope of this text; for information of this field, the reader is referred to some of the many books on the subject [9,38].

After the inclusion of natural quadrics and tori in a modeler, the next logical step would be to offer a blending operation. Recent relevant literature includes [55,84,101,56]. These advances seem to diminish the need for integrating parametric surfaces with solid modelers at least as far blending and fairing is concerned.

While the modeling space of natural quadrics and tori (and their blends) is quite adequate for mechanical parts, many other areas require the capability of processing true free-form parametric surfaces. Unfortunately, the integration of parametric surfaces into a solid modeler is a technically demanding task; for a good overview of the topic, see [40].

The approach followed in GEOMOD [118] is to bite the bullet and represent *all* geometric entities with rational B-splines. In this approach, a major problem is caused by the intersection curves of parametric surfaces, because in general they cannot appear as boundaries of surface patches. Instead, they must be represented approximately in the respective parameter spaces. For instance, the intersection curve of patches $P_1 = P_1(u_1, v_1)$ and $P_2 = P_2(u_2, v_2)$ would be represented by *two* curves $C_1 = C_1(u_1, v_1)$ and $C_2 = C_2(u_2, v_2)$.

Another, quite different approach to the integration of surface patches with solid models is to use an underlying polyhedral model and *smoothen* it by depositing surface patches that correspond with the faces, edges, and vertices of the polyhedron; this, for instance, is the approach chosen in MODIF [25].

Smoothing schemes must somehow deal with the problem of *anomalous regions* that appear at vertices of the original smoothed object. In principle, they would require nonrectangular surface patches. Unfortunately, no generally applicable families of n-sided patches, $n \neq 4$, have been presented in the literature. The solid modeler GEOMAP-III [64] avoids this difficulty by automatically subdividing anomalous regions into four-sided regions.

PROBLEMS

18.1. Based on the surface information of Program 18.1, describe an algorithm that can produce an image such as Figure 18.1(d).

18.2. Specify and implement the procedures for creating and deleting surface nodes and for assigning surface information to faces described in Section 18.1.2.

18.3. Specify and implement the procedures for creating and deleting curves and for assigning curve information to edges.

18.4. Based on the curve information of Program 18.2, outline an extended translational sweeping algorithm that assigns the surface information of the faces generated in the sweep.

18.5. Outline an extended rotational sweeping algorithm that assigns the surface information of the faces generated in the sweep.

18.6. Outline an improved ray casting algorithm for CSG models that makes use of hierarchical enclosures.

18.7. Outline an improved ray casting algorithm for boundary models that makes use of a 3-dimensional regular grid index.